Term Project Assignment

Comparing LSTM and Transformer Encoder Models for Text Classification

Task	Text classification
Dataset	AG News Classification
Models	LSTM classifier vs Transformer Encoder classifier
Training setting	Both models trained from scratch
Pretrained language models	Not allowed unless explicitly approved
Core outputs	Main comparison, one ablation, failure analysis, AI Usage Appendix

1. Project Overview

In this project, each team will compare an **LSTM classifier** and a **Transformer Encoder classifier** on the same text classification task. Both models must be trained from scratch.

The goal is not simply to achieve high accuracy. Students should analyze how recurrent sequence modeling and self-attention-based modeling differ in classification performance, training behavior, convergence patterns, failure cases, and sensitivity to experimental settings.

AI assistance tools such as ChatGPT, Copilot, Claude, Gemini, or similar tools may be used. Students are responsible for verifying all AI-generated code, explanations, results, and claims.

2. Learning Objectives

1. Implement and train an LSTM classifier.
2. Implement and train a Transformer Encoder classifier.
3. Compare the two models under controlled experimental conditions.
4. Evaluate models using accuracy, macro F1-score, loss curves, and confusion matrices.
5. Conduct one focused ablation study based on a clear hypothesis.
6. Analyze misclassified examples and explain model failures.
7. Use AI assistance tools responsibly and critically.

3. Dataset

3.1 Required Dataset

All teams should use AG News Classification. AG News is a standard four-class news classification dataset. Each input is a news text, and the task is to classify it into one of the following categories: World, Sports, Business, or Sci/Tech.

Recommended dataset links:

- Hugging Face ag_news
- TorchText AG_NEWS documentation
- Raw CSV access route

3.2 Dataset Access Rule

The AG News dataset may appear in several places, such as Hugging Face, TorchText, or raw CSV files. These are different access routes for the same benchmark dataset. Students should not mix multiple sources in one project.

Required rule: Use one dataset source only. The default recommended source is Hugging Face ag_news. Use TorchText or raw CSV files only if necessary. Do not mix Hugging Face, TorchText, and raw CSV versions in the same experiment.

3.3 Recommended Loading Method

```
from datasets import load_dataset

dataset = load_dataset("ag_news")
```

Students should create a validation split from the training set:

```
split_dataset = dataset["train"].train_test_split(test_size=0.1, seed=42)

train_data = split_dataset["train"]
valid_data = split_dataset["test"]
test_data = dataset["test"]
```

Important: Use the official test set only for final evaluation. Do not use the test set for model selection, hyperparameter tuning, or early stopping.

3.4 Dataset Documentation Requirements

- dataset source and access method
- train/validation/test split
- number of samples in each split
- number of classes and class distribution
- label mapping
- tokenization method
- vocabulary construction method
- padding strategy and truncation strategy
- maximum sequence length

Label indexing: Depending on the dataset source, labels may start from either 0 or 1. For PyTorch CrossEntropyLoss, labels should be integer class indices from 0 to num_classes - 1. For AG News, the final label range should be 0, 1, 2, 3.

4. Suggested Data Pipeline

1. Load AG News from one dataset source.
2. Create a validation split from the training set.
3. Inspect label values and confirm the label mapping.
4. Tokenize text.
5. Build vocabulary from the training data only.
6. Convert tokens to integer indices.
7. Pad or truncate sequences to a fixed maximum length.
8. Use the same processed inputs for both LSTM and Transformer Encoder models.

Avoid vocabulary leakage: Do not build the vocabulary using validation or test data.

5. Required Models

5.1 Model 1: LSTM Classifier

- embedding layer
- one-layer or two-layer LSTM
- final classification layer
- cross-entropy loss

Students should report vocabulary size, embedding dimension, hidden size, number of LSTM layers, dropout rate if used, pooling or final hidden-state strategy, and trainable parameter count.

5.2 Model 2: Transformer Encoder Classifier

- embedding layer
- positional encoding
- Transformer Encoder layers
- pooling or classification token strategy

- final classification layer
- cross-entropy loss

Students should report vocabulary size, embedding dimension, number of attention heads, number of Transformer Encoder layers, feedforward dimension, dropout rate, pooling or classification token strategy, and trainable parameter count.

Parameter count requirement: The two models do not need to have exactly the same number of parameters, but their sizes should be reasonably comparable unless the difference is explicitly justified.

Restriction on pretrained models: Pretrained language models such as BERT, DistilBERT, RoBERTa, MiniLM, or GPT-style models are not allowed unless explicitly approved by the instructor.

6. Recommended Starting Hyperparameters

Students may adjust these values, but all final settings must be reported.

Component	Suggested Starting Point
Maximum sequence length	128
Vocabulary size	10,000 to 20,000
Embedding dimension	128
Batch size	32 or 64
Optimizer	Adam
Learning rate	1e-3
Epochs	5 to 10
Dropout	0.1 to 0.3

Model	Suggested Starting Point
LSTM	1 or 2 layers; hidden size 128 or 256; bidirectional optional
Transformer Encoder	2 layers; 4 attention heads; feedforward dimension 256 or 512; sinusoidal or learned positional encoding; mean pooling or CLS token

7. Required Experiments

7.1 Main Comparison

Model	Role	Description
LSTM classifier	Baseline model	Recurrent sequence model trained from scratch
Transformer Encoder classifier	Comparison model	Self-attention-based sequence model trained from scratch

The comparison should use the same dataset split, preprocessing pipeline, vocabulary, maximum sequence length, evaluation metrics, and a similar training budget where possible.

7.2 Required Results

- accuracy
- macro F1-score
- training loss curve
- validation loss curve
- confusion matrix
- brief analysis of training stability and convergence behavior

A result table without interpretation is insufficient. Students must explain what the results suggest about the behavior of the two models.

8. Required Ablation Study

Each team must conduct one ablation study. The ablation must test a clear hypothesis and change one major factor at a time.

Ablation	Example Question
Sequence length: 128 vs 256	Does longer context improve performance?
Dataset size: 25%, 50%, 100%	Which model is more data-efficient?
Embedding dimension: small vs large	Does representation size affect performance?
Transformer layers: 1 vs 3	Does Transformer depth improve classification?
LSTM hidden size: small vs large	Does larger recurrent capacity help?
Dropout: with vs without dropout	Does regularization improve generalization?

For the ablation study, students should report the hypothesis, changed variable, controlled variables, result table, interpretation, and whether the result supports the hypothesis.

9. Failure Analysis

Students must analyze at least five misclassified examples. Where possible, the examples should include cases misclassified by both models, cases misclassified only by the LSTM, and cases misclassified only by the Transformer Encoder.

- input text
- true label
- LSTM prediction
- Transformer Encoder prediction
- confidence score if available
- possible reason for the error
- whether the two models failed similarly or differently

Possible causes to discuss include ambiguous wording, insufficient context, class overlap, truncation, tokenization issues, overfitting, underfitting, and model limitation.

Text	True Label	LSTM Prediction	Transformer Prediction	Possible Reason
Example news text	Business	Sci/Tech	Business	Contains technology-related terms but discusses market impact

10. Recommended Workflow

1. Load and inspect the dataset.
2. Build the tokenizer and vocabulary.
3. Train a very simple baseline such as average embedding plus linear classifier to check the data pipeline. This is recommended but not required for grading.
4. Train the LSTM classifier.
5. Train the Transformer Encoder classifier.
6. Run the required ablation study.
7. Generate evaluation figures and tables.
8. Conduct failure analysis.
9. Write the final report and prepare the presentation.

11. Common Implementation Pitfalls

Issue	Why It Matters
Label indexing	CrossEntropyLoss expects labels from 0 to num_classes - 1.
Vocabulary leakage	Vocabulary should be built from training data only.
Padding tokens	Padding should not dominate pooling or attention.

Sequence truncation	Important information may be removed if max length is too short.
Random seed	Results may vary without a fixed seed.
Train/eval mode	Use model.train() and model.eval() correctly.
Dropout	Dropout behaves differently during training and evaluation.
Device mismatch	Inputs, labels, and model must be on the same device.
Metric averaging	Macro F1 is important for class-level performance.
Test set usage	Use the test set only for final evaluation.

12. Fair Comparison Checklist

- Did both models use the same train/validation/test split?
- Did both models use the same tokenizer and vocabulary?
- Did both models use the same maximum sequence length?
- Were both models trained for a comparable number of epochs?
- Were the trainable parameter counts reported?
- Was the same metric used for both models?
- Was the ablation controlled, changing only one major factor?
- Were validation results used for model selection rather than test results?

13. Suggested Figures and Tables

13.1 Required

- table of model hyperparameters
- table of trainable parameter counts
- table of accuracy and macro F1-score
- training and validation loss curves
- confusion matrix
- table of ablation results
- failure analysis examples

13.2 Optional

- class-wise precision, recall, and F1-score
- attention visualization or token importance analysis
- confidence distribution
- learning curves under different dataset sizes
- error category breakdown

14. Suggested Interpretation Questions

1. Which model achieved better validation performance?
2. Did the better model also have better test performance?
3. Did one model converge faster than the other?
4. Did either model overfit?
5. Which classes were most frequently confused?
6. Did the two models fail on the same examples?
7. Did the ablation result match the hypothesis?
8. Was the Transformer Encoder more sensitive to data size, sequence length, or model depth?
9. Was the LSTM more stable under limited data?
10. What would be the most reasonable next experiment?

15. AI Usage Policy and Appendix

Students may use AI tools for code generation, debugging, concept explanation, visualization code, writing support, and hyperparameter suggestions. However, AI-generated code must be tested and understood, AI-generated explanations must be checked against course concepts, and students are responsible for all submitted code, figures, results, and claims.

AI Use	Example
Code generation	Asked an AI tool to draft a PyTorch training loop
Debugging	Used an AI tool to identify a tensor shape mismatch
Explanation	Asked an AI tool to explain positional encoding
Visualization	Asked an AI tool to generate confusion matrix code
Report editing	Used an AI tool to improve clarity of writing

Each team must submit an AI Usage Appendix of no more than one page. It should answer:

1. Which AI tools did you use?
2. What did you use AI for?
3. What AI-generated outputs were incorrect, incomplete, or misleading?
4. What did your team modify from the AI-generated output?
5. Which decisions were made by the team, not by AI?
6. How did AI use affect your project?

16. Deliverables

Deliverable	Description
Initial project plan	One page describing dataset split, preprocessing, model configuration, ablation plan, and expected analysis
Source code (Jupyter notebook file)	Clean, runnable, and reproducible code
Final report	4-6 pages, excluding AI Usage Appendix
AI Usage Appendix	Maximum 1 page
Final presentation	8-10 minute presentation plus Q&A

Recommended team size: 2-3 students.

17. Minimum Requirements and Optional Extensions

17.1 Minimum Requirements

1. Train one LSTM classifier.
2. Train one Transformer Encoder classifier.
3. Conduct one ablation study.
4. Report accuracy and macro F1-score.
5. Provide training and validation loss curves.
6. Provide a confusion matrix.
7. Analyze at least five misclassified examples.
8. Submit an AI Usage Appendix.
9. Submit final report, code, and presentation.

17.2 Optional Extensions

- additional ablation studies
- attention visualization or token importance analysis
- calibration analysis
- training cost comparison
- class-wise performance analysis
- detailed taxonomy of failure cases
- comparison of pooling strategies

- comparison of positional encoding methods

Optional extensions may strengthen the project but cannot replace the required components.

18. Grading Rubric

Criterion	Weight	Description
Problem definition and dataset understanding	10%	Clear task definition, dataset description, preprocessing, and split
Correct implementation and reproducibility	20%	Correct code, runnable pipeline, clear hyperparameters, reproducible results
Fair model comparison	15%	Comparable training conditions and appropriate evaluation metrics
Ablation study	15%	Clear hypothesis, controlled experiment, meaningful interpretation
Result interpretation	15%	Thoughtful analysis of metrics, curves, confusion matrix, and model behavior
Failure analysis	10%	Concrete analysis of misclassified examples and model limitations
AI usage transparency and verification	10%	Clear explanation of AI use, verification, corrections, and student decisions
Presentation quality	5%	Clear, concise, and well-organized presentation

Higher accuracy alone does not guarantee a higher grade. A project with moderate performance but careful experimental design and thoughtful analysis may receive a higher grade than a project with high accuracy but weak interpretation.

19. Final Report Structure

Section	Required Content
1. Introduction	Task description, motivation, and main research question
2. Dataset	Dataset description, source, label mapping, preprocessing, split, and class distribution
3. Models	LSTM architecture, Transformer Encoder architecture, parameter counts, and implementation details
4. Experiments	Main comparison, ablation study, training setup, and evaluation metrics
5. Results	Metric table, loss curves, confusion matrix, and convergence analysis
6. Failure Analysis	At least five misclassified examples and comparison of model failure patterns
7. Conclusion	Summary of findings, lessons learned, and possible future improvements
8. AI Usage Appendix	Tools used, how AI was used, what was verified or corrected, and which decisions were made by the team

20. Final Guidance

Students are encouraged to prioritize a clean, reproducible experimental pipeline over excessive model complexity. A simple but carefully controlled experiment with thoughtful analysis is preferable to a complicated experiment with unclear interpretation.